MURRAY STATE UNIVERSITY

Version control and git

How to collaborate and track code for your project

Tyler Stoffel

Fall 2026

Why Use Git?

- If you write code, you should use version control, even when you work alone.
- Git is free, works offline, and tracks ordinary files. You can use it without GitHub.[1]
- About 97% of professional developers answering Stack Overflow's 2022 version-control question reported using Git.[2]
- Recorded versions let you see exactly what changed and recover working code when a change causes problems.[1]
- Knowing Git makes you a more useful engineer. You can contribute to a team's code and leave work that someone else can continue.

1. Scott Chacon and Ben Straub. *Pro Git*, 2nd ed. Apress, 2014. Getting Started: What is Git?
2. Stack Overflow. 2022 Developer Survey: Version control systems. Professional Developers, 53,374 responses.

Example: Saving Versions of a Program

The RRO program works, but we want to try a change. To preserve the working code, we save another copy with a new filename.

```
rro_demo_v1.py
rro_demo_v2.py
rro_demo_final.py
rro_demo_final-fixed.py
```

- What changed between `v1` and `v2`?
- Which version worked when we tested the robot?
- What if we later save new changes over that working copy?

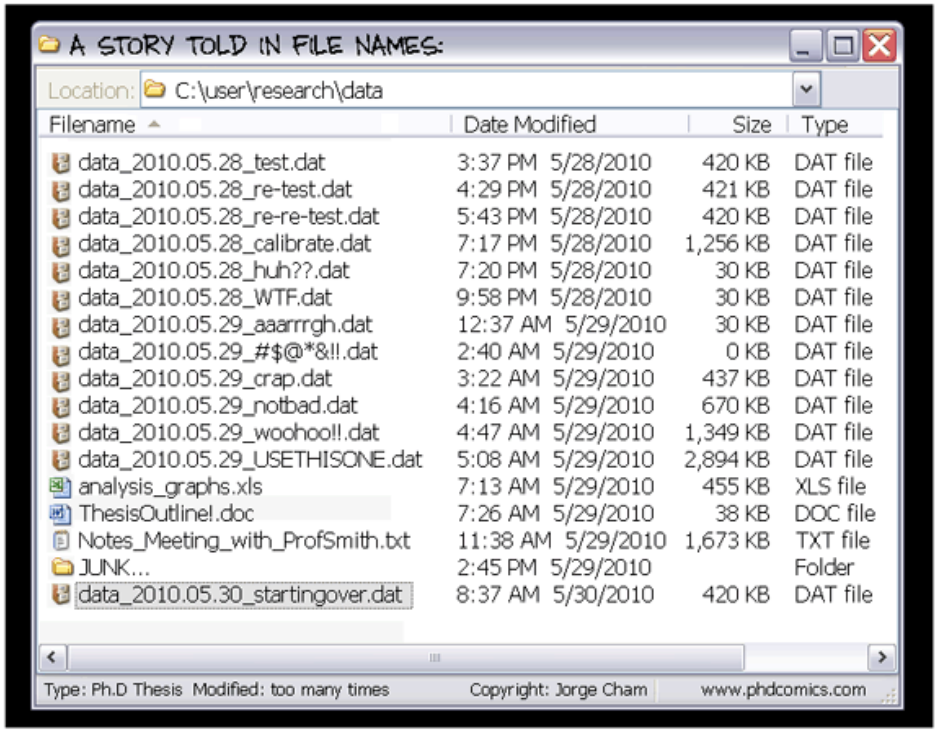


Figure 1: A story in file names, Jorge Cham, *PHD Comics*.

The filename alone does not tell us which lines changed, who changed them, or why.

Example: Collaborating on RRO Code

Two students copy `rro_demo_v2.py` and work separately. Student A changes the arm function. Student B changes the gripper function.

Student A: arm function

```
def extend_arm():
    print("Arm extended")
```

Student B: gripper function

```
def close_gripper():
    print("Gripper closed")
```

Each has a working program after several hours, but their copies are different.

- Replacing one whole file with the other can lose a student's work.
- Combining the files by hand can miss a change or bring back old code.
- If the combined program fails, which changes should we investigate?

Version control records the changes and their history so that we can compare versions, combine contributions, and recover recorded working code.

A **version control system** records revisions of files so that earlier states can be retrieved and compared.

Git is free, open-source software that runs on your computer and manages a project's file history. It is **distributed** because each full clone has its own repository and history.

- Compare changes to find what broke the code.
- Recover a recorded working version.
- Combine teammates' independently developed changes.

Linus Torvalds created Git in 2005 for Linux kernel development. Local work and commits can continue offline.



Figure 2: The Git logo.

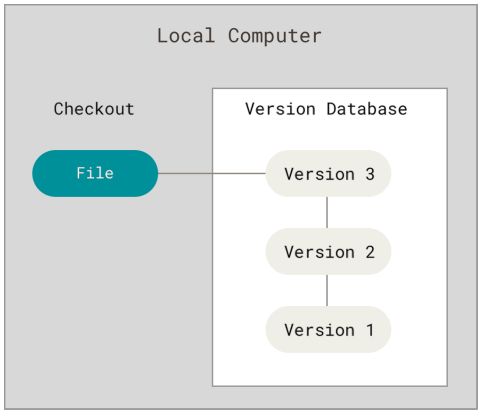


Figure 3: Files and recorded versions. Pro Git.

GitHub

GitHub is an online service that hosts Git repositories and provides tools for sharing and reviewing code.

- Our shared repository holds files and commit history online.
- Teammates exchange work through this shared copy.
- Pull requests support code review. Issues support discussion of bugs and tasks.

GitHub was founded in February 2008. Microsoft acquired it in 2018. Its 2025 report counted **more than 180 million developers**.

Today's materials: `github.com/tstoffel1`

`github.com/tstoffel1/egr498-git-tutorial`



Figure 4: Official GitHub logo.

GitHub Desktop

GitHub Desktop is an application that provides a graphical interface for Git operations and exchanging code with GitHub.

- **Changes** shows saved edits and lets us create a commit.
- **History** lets us inspect earlier commits.
- Repository, branch, and synchronization controls manage where we work and share changes.

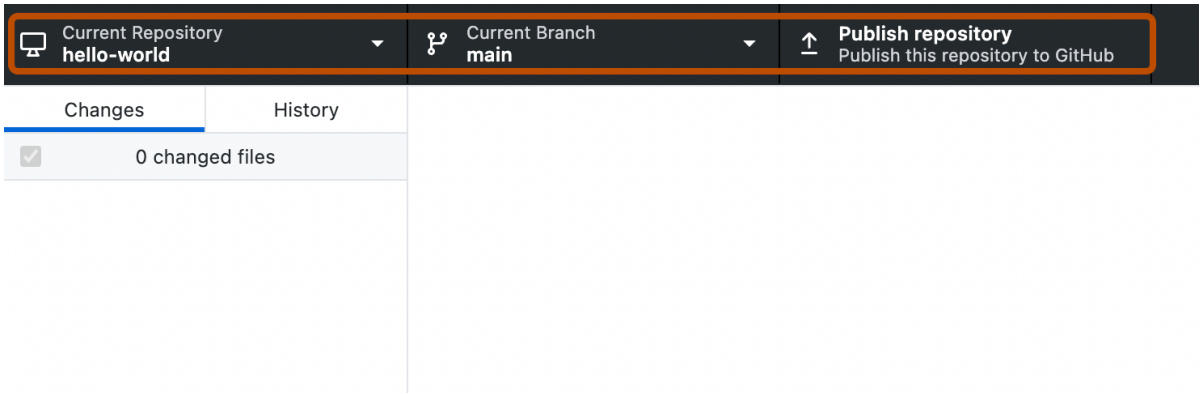


Figure 5: Desktop's repository, branch, and publishing controls. GitHub Docs.

We edit and run Python in an editor such as **Thonny** or **VS Code**. GitHub Desktop runs on Windows/macOS. On Raspberry Pi OS, use Git in the terminal.

Repository and Commit

A **repository** stores the version-controlled project content and its history.

The **working copy** (working tree) contains the files checked out for editing.

The **staging area** holds file content selected for the next commit. **Stage changes** selects that content.

A **commit** records a project snapshot in the **local repository** on your computer, using the prepared staging area. **Commit changes** performs this operation.

Each commit has an identifier, author, message, and parent links.

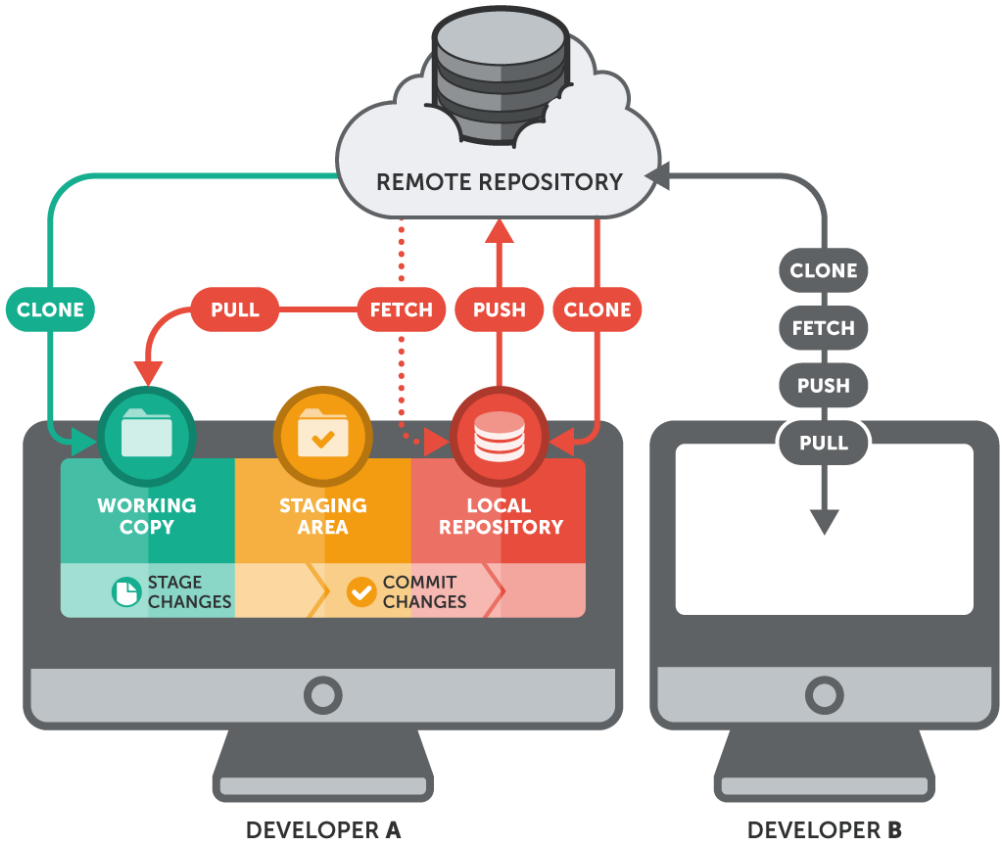


Figure 6: Working files and recorded history on Developer A's computer. Tower.

Clone, Push, Fetch, and Pull

A **remote repository** is another repository with which we exchange revisions. Our team's GitHub repository is the shared remote for Developers A and B.

Clone creates a local repository and working copy from an existing repository.

Push sends local commits to the remote and requests a branch update.

Fetch downloads remote history. It leaves our working files unchanged.

Pull fetches remote history and integrates it into our current branch, updating the working files.

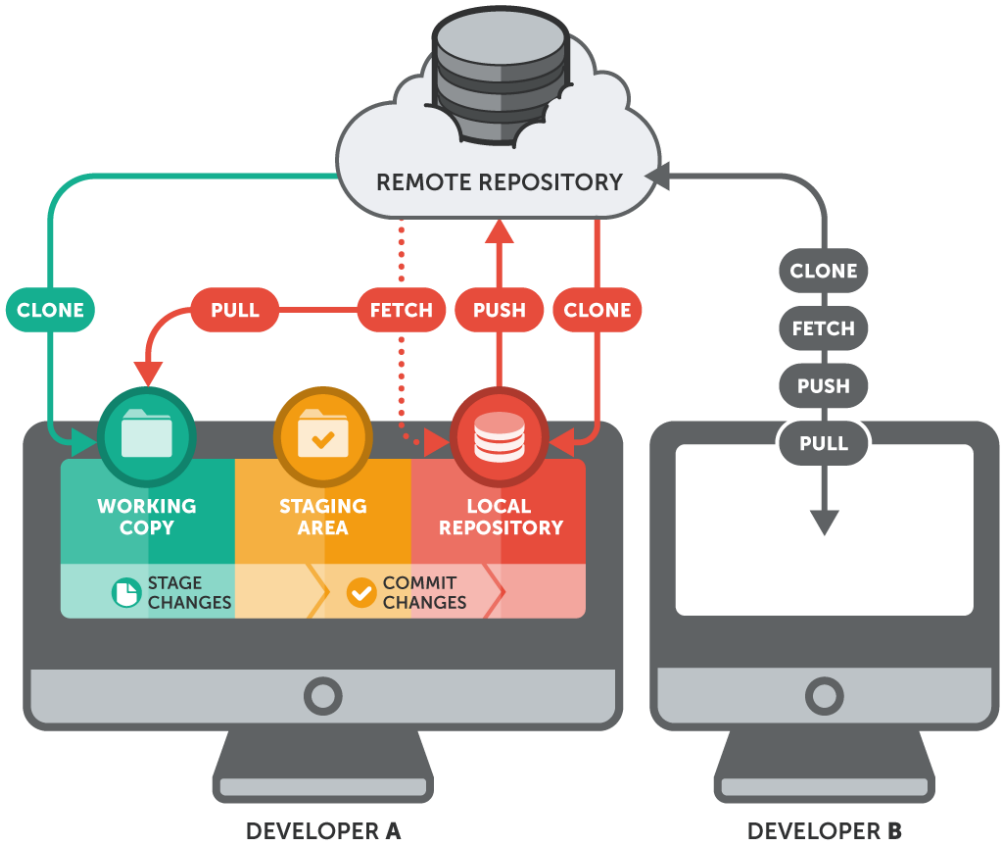


Figure 7: Developers A and B exchange commits through the remote repository. Tower.

Branches

A **branch** is a name that points to the latest commit in a line of development. Creating a branch lets us record separate changes from a shared starting commit. A **feature branch** holds work on a particular change.

- GitHub normally starts a new repository on `main`. Local `git init` uses a configurable name. Git 2.x defaults to `master`.
- Desktop's **Current Branch** menu shows and selects the branch. Switching branches updates the working files to that branch's version.

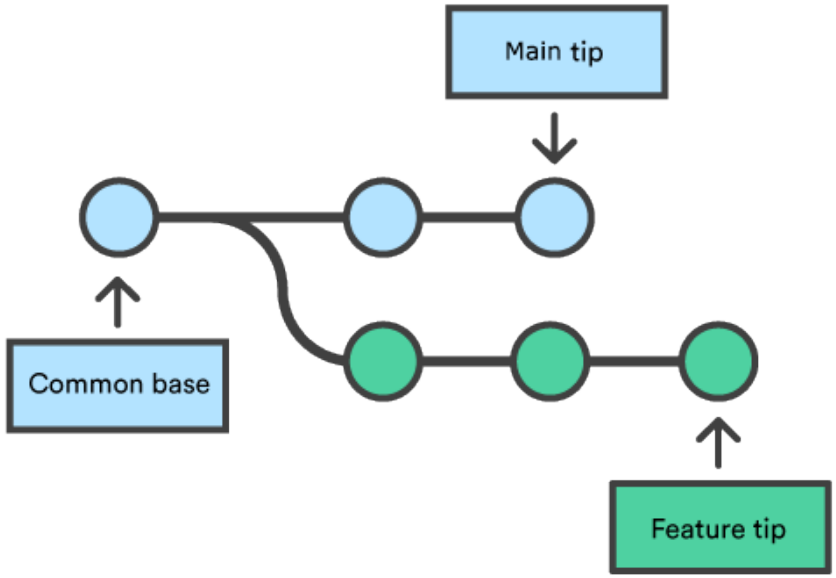


Figure 8: Each circle is a commit. The **common base** is the shared starting commit. A **tip** is the latest commit on a branch. Atlassian.

Merging

A **merge** integrates another branch's changes and history into the current branch. Here, both branches have new commits. Git creates a **merge commit** with two parent commits, preserving both histories.

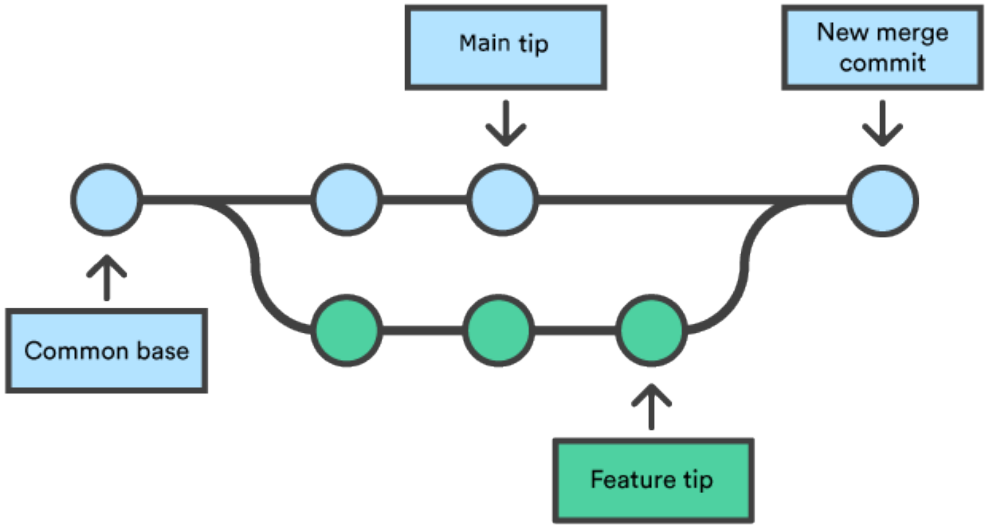


Figure 9: Merge the green feature branch into the blue `main` branch. After the merge, `main` points to the new merge commit. Atlassian.

- For the RRO, we can combine arm and gripper changes on `dev` using this process.
- A **merge conflict** is an edit Git cannot combine automatically. We resolve it, then test the combined program.

Tutorial Setup

Open `github.com/tstoffel1/egr498-git-tutorial`. The repository's README identifies the files to open. `DEMO-STEPS.txt` contains the complete workflow.

1. Choose **Code**, then **Open with GitHub Desktop** to clone it.
2. Select `demo` in Desktop's **Current Branch** menu. Make all tutorial edits on `demo`.
3. Open `rro_demo.py` in a Python editor.

```
egr498-git-tutorial/
  README.md              (what to open)
  DEMO-STEPS.txt         (teaching sequence)
  PROJECT.md             (first edit)
  rro_demo.py            (Python program)
  slides/                (presentation)
```

Open `PROJECT.md` for the first example, then edit `rro_demo.py`. The starter is already committed. Here, `demo` serves the role of your team's `dev` branch. We will make and share our changes in this repository. Students can fork it to push practice work to their own accounts.

Git Tracks Ordinary Files

Git tracks ordinary files in a repository folder. We edit those files in a text editor or a Python IDE. Start with the committed `PROJECT.md`, add the second line, and click **Save**.

```
# RRO software
We will test the arm function.
```

```
$ git status --short
 M PROJECT.md
```

Status reports the state of files. `M` means the tracked file is **modified**. Desktop shows the same saved edit in **Changes**:

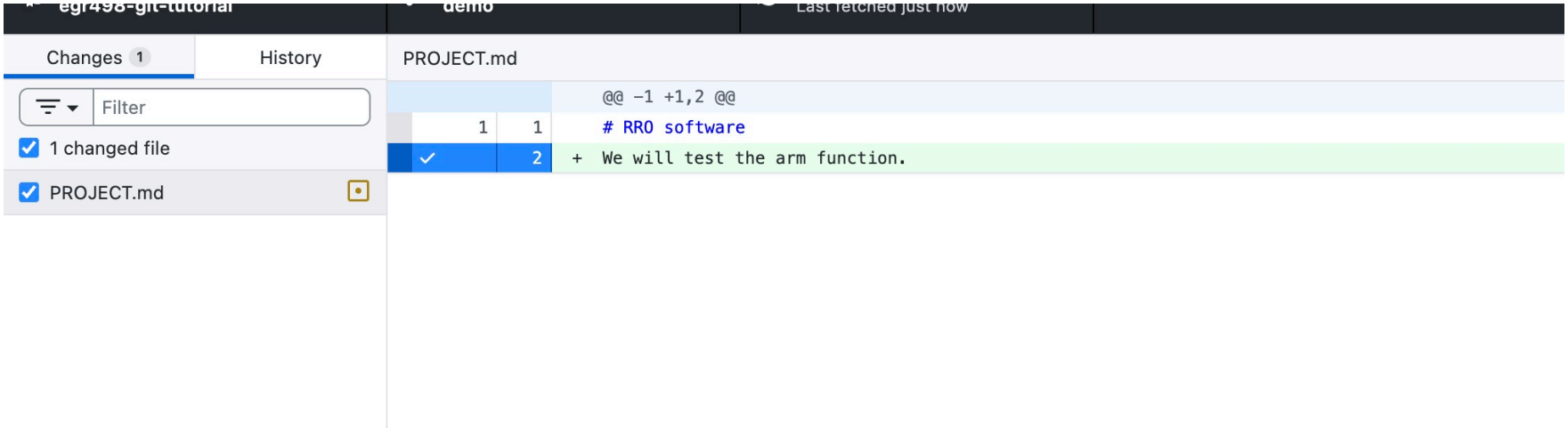


Figure 10: The added line in our tutorial repository.

With autosave disabled, an unsaved edit exists only in the editor. After saving, Git detects it. A **commit** records the new revision in history.

Python Program

Our Raspberry Pi program calls the arm and gripper functions, then prints three status updates. The functions initially print `not configured`.

```
from time import sleep

POLL_INTERVAL_S = 0.5  # s between status updates

def main():
    extend_arm()
    close_gripper()
    for update in range(1, 4):
        print(f"Status update {update}")
        sleep(POLL_INTERVAL_S)


if __name__ == "__main__":
    main()
```

Save the file and run it from the repository folder:

```
python3 rro_demo.py
```

The same file runs on a laptop or Raspberry Pi with Python 3. Printed messages represent robot actions in this demonstration.

Committing Changes

1. In **Changes**, select `PROJECT.md` and inspect the added line.
2. Check the file to include it in this commit.
3. Enter **Describe the arm test**, then click **Commit to demo**.

The **staging area** holds content selected for the next commit. Desktop manages this selection through its commit controls.

Open **History**. Our edit now has a recorded commit. Click **Publish branch** to share this branch on our GitHub repository.

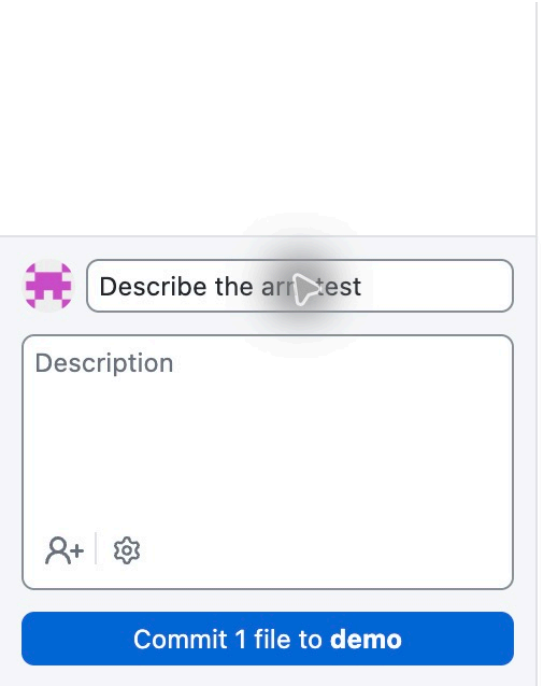


Figure 11: Our message and commit control in Desktop.

Modifying the Program

A **diff** shows the differences between file versions. Change the status-update interval from 0.5 s to 0.1 s.

```
- POLL_INTERVAL_S = 0.5  # s between status updates
+ POLL_INTERVAL_S = 0.1  # s between status updates
```

1. Save and run `python3 rro_demo.py`. The status messages now appear more quickly.
2. In **Changes**, inspect the diff. Only the intended timing value should change.
3. Commit as **Set status interval to 0.1 s**, then **Push origin**.

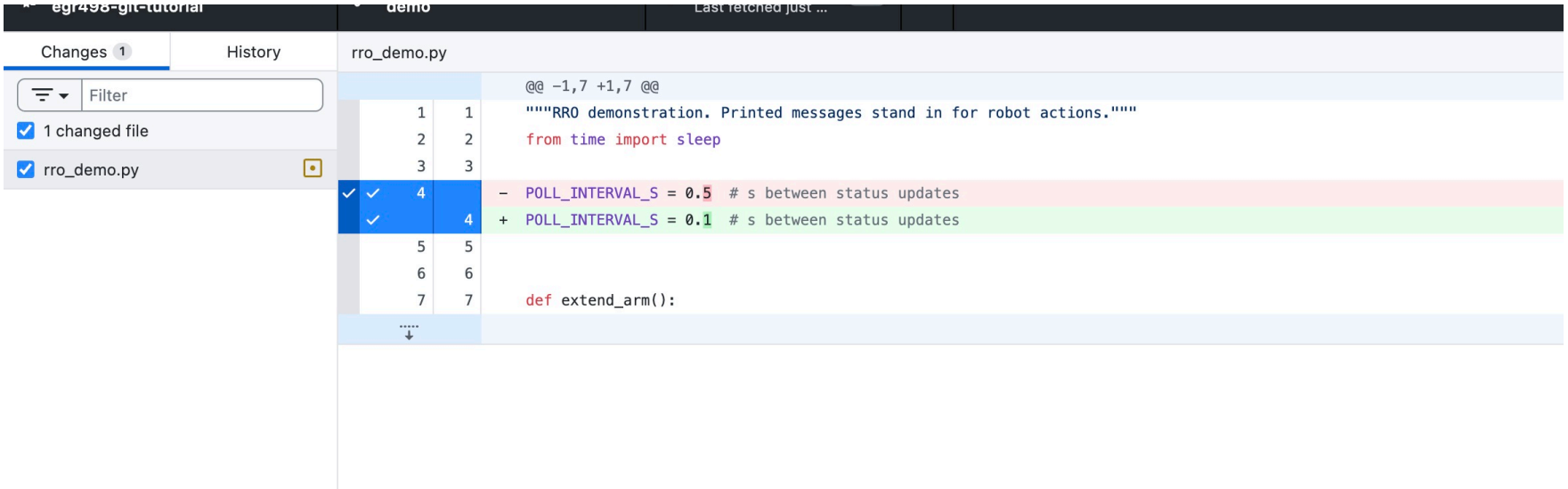


Figure 12: The recorded Python change from 0.5 to 0.1 seconds.

Commit History

A **log** lists recorded commits and their descriptive information.

1. Click **History**, then select **Set status interval to 0.1 s**.
2. Read the message and commit identifier. The diff shows exactly which line changed in this commit.

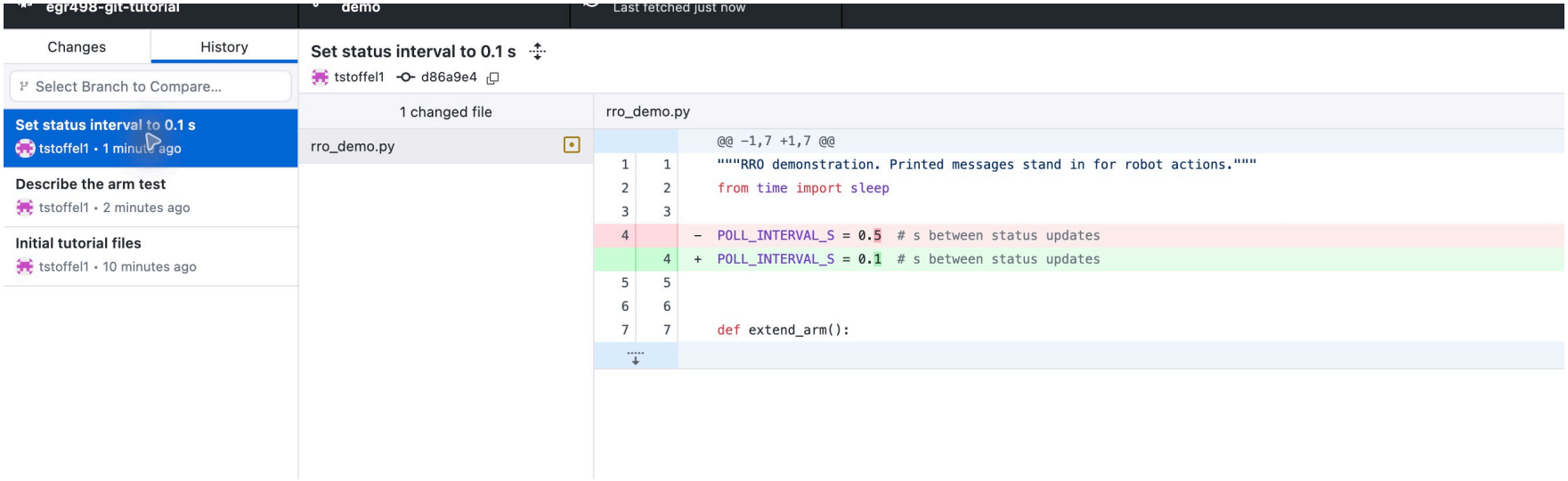


Figure 13: The timing commit and its recorded diff in Desktop.

Selecting an entry displays information. It leaves the files on disk unchanged. The command equivalents are `git log` and `git show`.

Reverting a Commit

Set the interval to -0.1 s, save, and commit as **Test negative status interval**.

```
POLL_INTERVAL_S = -0.1  # s between status updates
```

Run the program. Python reports an error:

```
ValueError: sleep length must be non-negative
```

A **revert** creates a new commit that reverses the changes introduced by a selected commit.

1. In **History**, right-click the negative-interval commit.
2. Choose **Revert Changes in Commit**.
3. Confirm the interval is again 0.1 s. Run the program successfully, then push.

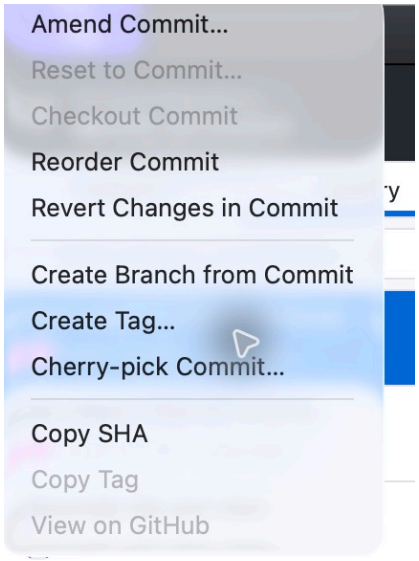


Figure 14: Revert records the reversal.

Developing on a Branch

Push the recovered `demo` branch to GitHub. We now represent Student A, who is changing the arm function.

1. From `demo`, create `arm-feature` in **Current Branch**.
2. Replace the line inside `extend_arm()` in the active Python file:

```
-     print("Arm: not configured")
+     print("Arm extended")
```

Save, run the program, and commit as **Add the arm demonstration**.

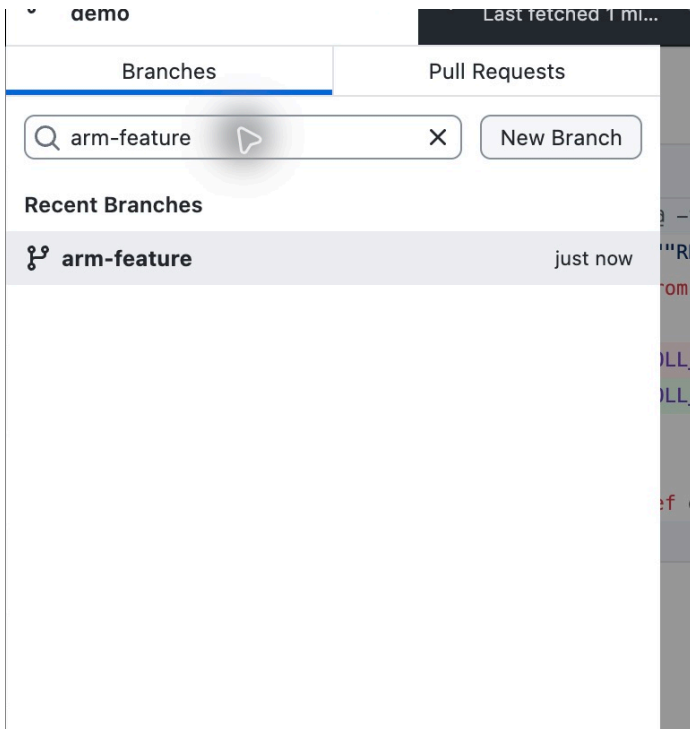


Figure 15: Our development branches in Desktop.

Our arm change is on `arm-feature`. Switching back to `demo` shows the unchanged arm function.

Pulling a Teammate's Changes

Student B changes the gripper function in the shared `demo` branch. We simulate this by editing the Python file **on GitHub**, using the same instructor account.

```
-     print("Gripper: not configured")
+     print("Gripper closed")
```

1. On GitHub, select `demo`, open `rro_demo.py` in the repository's top folder, and click the pencil. Save the edit with **Commit changes**, directly to `demo`.
2. In Desktop, switch to `demo`. Click **Fetch origin**, then **Pull origin**.

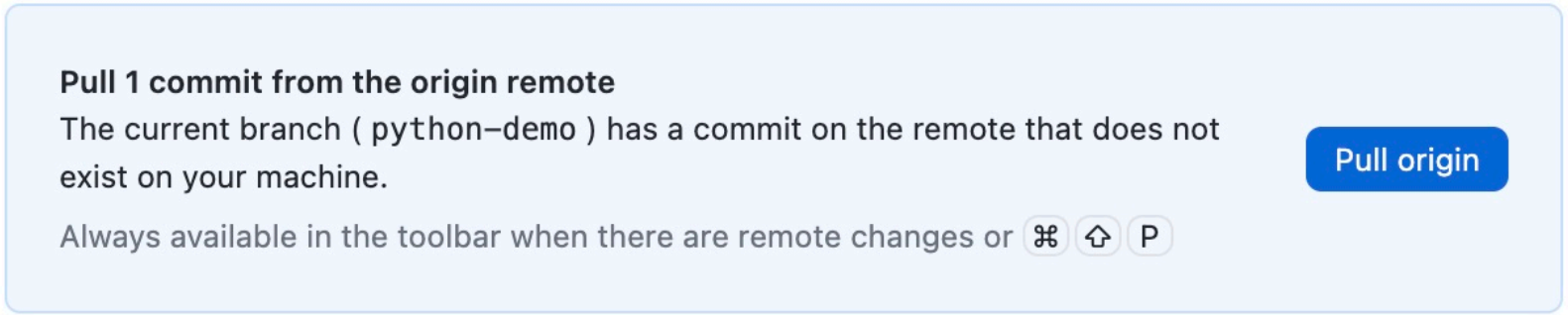


Figure 16: Desktop reports one remote commit ready to pull.

The local Python file now has B's gripper change. A's arm change is still on `arm-feature`.

Merging Code Changes

Stay on `demo`, which now contains the gripper change. Choose **Branch**, then **Merge into current branch**. Select `arm-feature` and merge it into `demo`.

The combined file contains both contributions:

```
def extend_arm():
    print("Arm extended")

def close_gripper():
    print("Gripper closed")
```

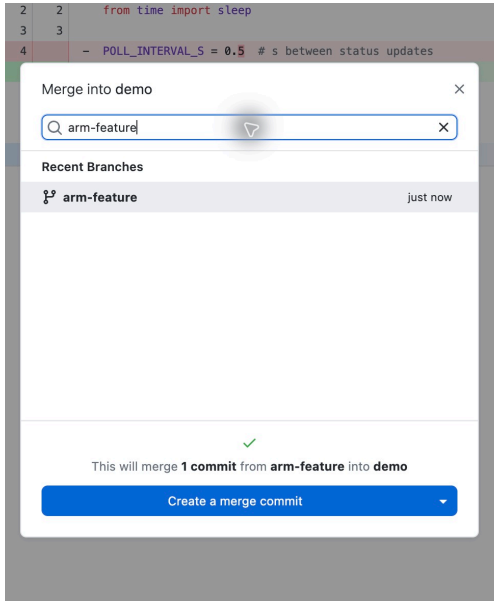


Figure 17: Merge the arm branch into `demo`.

Run the combined program, then **Push origin**. The output begins with `Arm extended` followed by `Gripper closed`. Git combined different edits in the same file. We still test the program.

Running the Code on a Raspberry Pi

GitHub Desktop manages the laptop's repository. The Raspberry Pi needs its own copy of the files and history.

Clone the repository onto the Pi once, then select the branch to run. Inside that repository, receive the latest commits and run the program:

```
git pull --ff-only
python3 rro_demo.py
```

Use `dev` for your team's development code. Select the same development branch on the Pi before pulling and running it.

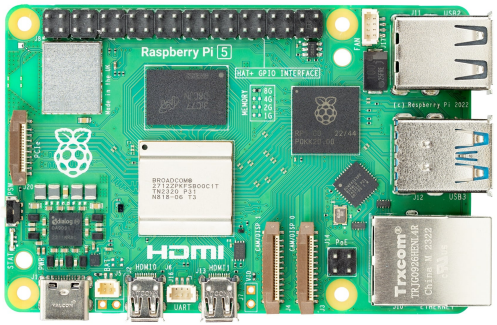


Figure 18: A Raspberry Pi 5. Raspberry Pi Ltd.

Pushing a commit from a laptop does **not** automatically update the Pi's files or restart its program. Pull on the Pi, then run and test the updated code.

Team Repository Access

Each team has its own repository with two branches.

- **dev**: the version everyone works on. Students save, commit, and push their changes here.
- **main**: the version the software coordinator has reviewed and tested. Everyone can read it, but only the coordinator(s) can update it.

Your coordinator invites your GitHub account to the team repository. Accept that invitation to get permission to contribute.

For this assignment, you will push directly to `dev`. You do **not** need to create a pull request. The coordinator reviews the team's changes and merges them into `main`.

The coordinator sets GitHub's branch permissions to protect `main`. The branch names alone do not control who can change them.

Assignment: Installation and Access

Complete this assignment **after the guest lecture**. Start here, even if you have never used GitHub.

1. **Install GitHub Desktop** on your Windows or Mac computer from `desktop.github.com`.
2. **Create an account on GitHub** at `github.com/signup`, or use your existing account.
3. **Email your software coordinator** your name, Blue or Gold team, role number, and **GitHub username**.
4. **Wait for the invitation, then accept it.** Open GitHub's invitation email while signed into that account.
5. **Open GitHub Desktop and sign in** with that same GitHub account. Follow the browser prompts back to Desktop.

Sending your username lets the coordinator invite you. Accepting the invitation completes the access step.

Assignment: Open the Team Files

Blue: `mmurphy39-hash/Senior-Design-Blue`
Gold: `BraxtonAlexander0/Senior-Design-Gold-Test`

6. In Desktop, click **File**, then **Clone Repository**, then the **URL** tab. Paste your team's link and click **Clone**. This downloads the project to your computer.
7. Click **Current Branch** at the top. Find and click the existing **dev** branch. Check that the toolbar now says **dev**. This selects, or *checks out*, that branch.
8. Click **Fetch origin**. If **Pull origin** appears, click it to get the latest team changes.

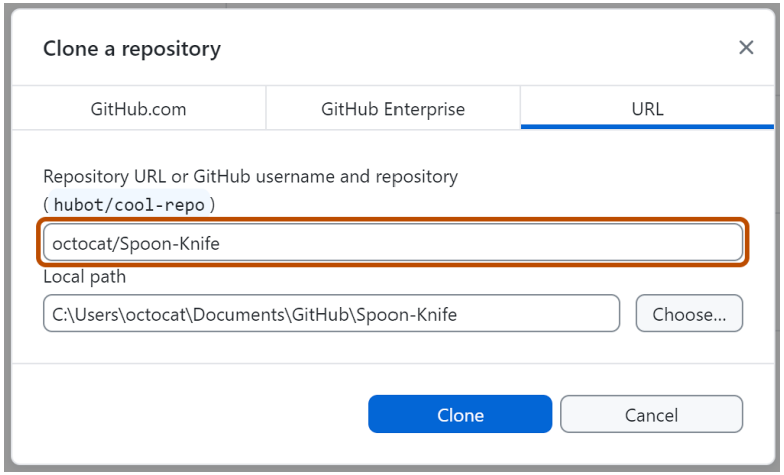


Figure 19: Paste your team's link above into the URL field. GitHub Docs.

Accept your repository invitation before contributing. Use the existing `dev` branch.

Assignment: Team Roster

Each team has a `team-roster.txt` file with 21 entries. Here are the first six entries for Blue. Long descriptions wrap in this view.

Role	Role description	Technical assignment description	GitHub username	Hometown
1	Team Leader	Common Sweeper/MMP Mobility Platform		Lafayette, LA
6	Mobility Systems Engineer	Sweeper/MMP Drivetrain & Terrain Mobility		Columbia, MO
2	Systems Integration Coordinator	MCO/RRO High-Mobility Platform		Omaha, NE
20	Cost & Logistics Coordinator	MCO Acquisition, Retention & Transport System		Sioux Falls, SD
18	Test & Measurement Coordinator	MCO Mass Measurement & Classification System		Duluth, MN
7	Recovery & Manipulation Engineer	RRO Reach, Manipulation & End-Effector System		Fresno, CA

Find your **role number**. Add your username in the blank column and replace the example hometown with your own. Keep the other information as it is.

Assignment: Edit and Share Your Row

9. Click **Repository**, then **Show in Explorer** (Windows) or **Show in Finder** (Mac). Open `team-roster.txt` in a plain text editor. Find your **role number**, fill in your GitHub username, correct your hometown, and **Save**. Edit only your row.
10. Return to Desktop. Click **Changes**, then `team-roster.txt`. Check the red and green text to see exactly what you changed.
11. Check the box beside the file. Enter a **Summary**, such as **Add username and hometown for role 7**. Click the blue commit button ending in **to dev** to record your change locally.
12. Click **Fetch origin**, then **Pull origin** if offered. Click **Push origin** to send your commit to the team.
13. Click **Repository**, then **View on GitHub**. Select **dev** on the website and open the roster. Confirm your row is correct online.

```
  Role   GitHub username      Hometown
- 7                          Fresno, CA
+ 7      example-student     Murray, KY
```

If Desktop reports a conflict or an access problem, ask your coordinator for help.

Brief numbered assignment instructions: **Blue README** and **Gold README**.